

AI PDLC Competency Matrix v2

Эталонная матрица компетенций для AI PDLC Gap-Analyzer

Школа 21 (Сбер) — рабочий черновик, версия 2.0 (program-first)

Статус: черновик v2 (program-first). Заменяет v0.1.

Архитектура матрицы оценивает образовательные программы — какие AI PDLC-компетенции и на каком уровне формирует программа у выпускника. Профиль конкретного человека — производный слой (Часть 4).

Часть 1. Методология (коротко и прикладно)

Program-first. Матрица оценивает не человека, а образовательную программу: какие AI PDLC-компетенции и на каком уровне формирует программа у выпускника.

Человек/выпускник — производный слой (Часть 4): его профиль строится как проекция через те же 6×10 ячеек, но через индивидуальные артефакты вместо программных.

Основной вопрос модели: *Какие AI PDLC-компетенции и на каком уровне формирует программа?*

Уровни владения: N0 → J1–J3 → M1–M3 → S1–S3 (10 уровней на компетенцию)

Группа	Уровни	Логика	Ключевой глагол
N0	—	обязателен комментарий: чего не хватает для перехода к J1	—
Junior	J1–J3	Ознакомительный/начальный	использует / пробует / знакомится
Middle	M1–M3	Уверенное применение	управляет / интегрирует / оптимизирует
Senior	S1–S3	Системное и глубокое использование	проектирует / масштабирует / систематизирует

Conservative extension of Sber PDLC

Мы не создаём новую модель компетенций. Сохраняем: названия 6 компетенций, прогрессию Junior → Middle → Senior, maturity logic и формулировки из скрина Сбера (они становятся «якорными» формулировками внутри J/M/S-диапазонов — обычно ближе к верхнему уровню, например формулировка скрина для Senior ≈ S2).

Усиливаем: явные observable behavior (знает/умеет/применяет), 4-слойную evidence-модель, применимость к оценке программы (program assessment applicability), детализацию диапазона на 3 подуровня там, где у Сбера была одна ступень.

Evidence model (4 слоя)

Определяет, насколько глубоко компетенция закреплена, независимо от формулировки уровня.

Уровень evidence	Тип
E1	Теория / sandbox
E2	Есть в проекте
E3	Повторяется в ≥2-3 проектах
E4	Reusable / используют другие

Evidence особенно критичен для Senior: формулировки вида «умеет сделать tool» недостаточны для S-уровня без E4 («создал reusable tooling, используемый повторно другими»).

Часть 2. Полная AI PDLC Competency Matrix v2

6 компетенций × 10 уровней (N0 + J1–J3 + M1–M3 + S1–S3). Для каждого уровня: Знает / Умеет / Применяет (observable behavior), Evidence (целевой Е-уровень) и Критерий перехода на следующий уровень.

Якорные формулировки из скрина Сбера выделены цветом и помечены «(якорь Сбера)» — они зафиксированы как есть и расширены вокруг себя на J1–S3. Подробные таблицы по каждой компетенции — на следующих страницах (альбомная ориентация).

2.1 Prompting

Формирование намерения и постановка задач агенту

Level	Знает	Умеет	Применяет	Evidence	Критерий перехода
N0	—	—	Программа не содержит заданий на формулирование запросов к ИИ.	—	Появление хотя бы одного задания, где студент формулирует запрос к ИИ с заданным форматом результата.
J1	Что такое промпт, зачем нужен контекст и формат ответа в запросе.	Формулирует простой одношаговый запрос.	В учебном задании формулирует запрос к ИИ-инструменту и получает ответ.	E1	Появляется требование очистки запроса от чувствительных данных и оценка качества ответа.
J2	Какие данные нельзя передавать в запрос (ПДН, секреты); признаки нерелевантного ответа.	Очищает запрос от чувствительной информации; распознаёт нерелевантный/некорректный ответ.	Очищает запрос перед отправкой; отмечает, что ответ ИИ не подходит.	E1-E2	Появляется требование переформулировать запрос после неудачи, с фиксацией изменений.
J3	Принципы итеративного уточнения запроса.	Переформулирует запрос по итогам неудачной попытки, а не повторяет тот же.	(якорь Сбера, базовый слой) Делает простые постановки задач, очищает запросы от чувствительной информации.	E2	Появляется задача в «агентном режиме»: постановка с границами + проверка результата + решение.
M1	Что такое границы задачи для агента и зачем их задавать явно.	Ставит задачу агенту с явными границами.	В проекте ставит агенту задачу с описанием допустимых изменений.	E2	Появляется ручная проверка результата перед принятием как обязательный шаг.
M2	(якорь Сбера) Принципы постановки задач агенту, проверки результата, корректировки и принятия решения от агента.	Проверяет результат работы агента перед принятием; корректирует и принимает решение «принять/доработать/отклонить».	(якорь Сбера) Делает постановки задач в агентном режиме, проверяет результат, корректирует и принимает решения от агента.	E2-E3	Критерии принятия результата формализуются (мини human-in-the-loop Decision Map), повторяются в нескольких проектах.
M3	Как формализовать критерии принятия результата агента (мини Decision Map).	Применяет единые критерии принятия результата в нескольких задачах подряд.	В >=2-3 проектах использует одну и ту же схему «постановка -> проверка -> решение».	E3	Появляется цепочка промптов (multi-step) и few-shot примеры с обоснованием.
S1	Принципы построения цепочек промптов (multi-step) и few-shot prompting.	Строит цепочку промптов для сложной задачи, обосновывает разбивку; подбирает few-shot примеры.	(якорь Сбера) Использует продвинутые шаблоны (цепочки промптов, few-shot learning) в проекте.	E3	Шаблон промпта/цепочки оформляется как переиспользуемый артефакт.
S2	Принципы оформления промпта как переиспользуемого артефакта (библиотека промптов).	Оформляет промпт/цепочку как шаблон с описанием назначения для других пользователей.	Создаёт шаблон промпта, который можно передать другому человеку без доработки.	E3-E4	Шаблон фактически переиспользован другим участником команды/группы.
S3	(якорь Сбера) Принципы ведения и переиспользования командной библиотеки промптов.	Поддерживает и пополняет общую библиотеку промптов команды/программы.	(якорь Сбера) Ставит задачи агенту и пополняет библиотеку промптов (промпты переиспользуются другими).	E4	— (высший уровень компетенции)

Закрашенные строки/ячейки — **якорные формулировки из исходной матрицы Сбера**; N0 — отсутствие покрытия.

2.2 Prompt Engineering

Управление правилами работы агента

Level	Знает	Умеет	Применяет	Evidence	Критерий перехода
N0	—	—	Программа не упоминает rules-файлы/системные промпты как управляемый артефакт.	—	Появление задания «используй готовый rules-файл перед началом работы».
J1	Что такое rules-файл/системный промпт и зачем он нужен.	Находит и применяет готовый rules-файл перед началом сессии.	Использует выданный преподавателем/программой rules-файл в задании.	E1	Появляется требование объяснить назначение конкретных правил из файла.
J2	Какие риски/ошибки предотвращают конкретные правила в rules-файле.	Объясняет назначение 1-2 правил из готового rules-файла.	(якорь Сбера) В начале каждой сессии применяет правила и ограничения (rules) для работы агента.	E1-E2	Появляется требование не обходить ограничения и фиксировать это в рефлексии.
J3	Принцип «ограничения rules существуют не для замедления, а для снижения риска».	Соблюдает rules даже когда это снижает скорость; фиксирует попытки и результат.	Применяет неизменённый rules-файл в реальном/курсовом проекте.	E2	Появляется задача редактирования/создания собственного rules-файла под проект.
M1	Структура rules-файла; что в нём можно настраивать под задачу.	Редактирует/создаёт rules-файл под конкретную задачу/проект.	В проекте создаёт собственную версию rules-файла.	E2	Появляется plan mode: агент предлагает план, человек одобряет до выполнения.
M2	(якорь Сбера) Принцип plan mode: агент сначала формирует план/ТЗ, человек проверяет до выполнения.	Использует plan mode для постановки сложной задачи (план -> одобрение -> выполнение).	(якорь Сбера) Использует rules как постоянный управляемый артефакт; применяет агента для написания технического задания (plan mode) при решении комплексных задач.	E2-E3	Появляется версионирование rules (история изменений с причинами), повторяющееся в нескольких проектах.
M3	Принципы версионирования управляемых артефактов (rules) с фиксацией причин изменений.	Ведёт версии rules-файла с указанием, что и почему изменено, в >=2-3 проектах.	Поддерживает версионированный rules-файл на протяжении нескольких итераций проекта.	E3	Появляется rules-файл/системный промпт командного уровня, используемый несколькими людьми.
S1	Принципы формулирования rules для командного/группового использования.	Разрабатывает rules-файл/системный промпт, рассчитанный на использование другими.	Создаёт rules-файл командного уровня для своей учебной группы/проектной команды.	E3	Появляется процесс актуализации rules при изменении контекста проекта.
S2	Принципы поддержания актуальности rules (review-цикл, триггеры обновления).	Поддерживает и пересматривает rules при изменении контекста проекта.	Обновляет командный rules-файл по мере изменения проекта, фиксирует это.	E3-E4	Появляется обоснование trade-off в формулировках правил через категории риска.
S3	(якорь Сбера) Принципы баланса между свободой агента и риском ошибки при формулировании правил командного уровня.	(якорь Сбера) Разрабатывает и актуализирует rules-файлы и системные промпты командного уровня; команда использует актуальные настройки агентов.	Объясняет 1-2 ключевых правила через категории риска; rules используются всей командой.	E4	— (высший уровень компетенции)

Закрашенные строки/ячейки — **якорные формулировки из исходной матрицы Сбера**; N0 — отсутствие покрытия.

2.3 Context Management

Формирование контекста для агента

Level	Знает	Умеет	Применяет	Evidence	Критерий перехода
N0	—	—	Программа не содержит заданий на сбор/подготовку контекста для агента.	—	Появление задания «прикрепи к запросу нужные файлы».
J1	Что входит в контекст запроса (файлы, фрагменты, формулировка).	Прикрепляет файлы/фрагменты к запросу.	Выполняет задание, прикрепляя материалы к запросу агенту.	E1	Появляется требование отбирать релевантные материалы, а не прикреплять «всё».
J2	Принцип «минимально достаточный контекст» (релевантность vs объём).	Отбирает релевантные файлы/фрагменты, избегая избыточности.	(якорь Сбера) Сборка контекста в момент запроса: специалист дополняет контекст по запросу агента, прикрепляет необходимые файлы, отвечает на вопросы.	E1-E2	Появляется требование проверки на отсутствие чувствительных данных в контексте.
J3	Какие данные не должны попадать в контекст (ПДН, секреты) и как это проверить.	Проверяет контекст на отсутствие чувствительной информации перед отправкой.	(якорь Сбера) Не передаёт чувствительную информацию в контекст агента.	E2	Появляется задача оформления контекста как структурированного артефакта (markdown-спека/ТЗ).
M1	Что такое структурированная спецификация контекста (markdown-спека, ТЗ) и зачем она нужна.	Оформляет контекст задачи как структурированный артефакт, а не свободный текст.	В проекте создаёт markdown-спецификацию/ТЗ как контекстный пакет для агента.	E2	Появляется осознанное ограничение набора подключённых инструментов/скиллов под задачу.
M2	(якорь Сбера) Принцип «контекстный пакет не перегружен»: управление harness, отказ от лишних скиллов/тулзов.	Осознанно ограничивает набор подключённых инструментов/скиллов; формирует структурированные артефакты под задачу.	(якорь Сбера) Контекстный пакет собран под решение задачи: управление harness (не использует лишних скиллов/тулзов), создаёт структурированные артефакты, включая markdown-спеки и md-документы.	E2-E3	Появляется умение сворачивать/суммировать длинный контекст, сохраняя ключевые факты.
M3	(якорь Сбера) Принципы сворачивания и суммирования контекста длинной сессии без потери ключевых фактов.	Сжимает/суммирует длинный контекст сессии, фиксируя, что сохранено и что отброшено.	(якорь Сбера) Сворачивание и суммирование контекста в рамках задачи, повторяется в >=2-3 проектах.	E3	Появляется задача определения «обязательного контекста» для команды/продукта.
S1	Принципы выделения «обязательного контекста» команды/продукта: документация, гайдлайны, архитектурные решения.	Определяет состав обязательного контекстного слоя для команды/программы.	(якорь Сбера) Формирует общий контекстный слой команды/продукта: определяет состав документов, спецификаций, гайдлайнов, архитектурных решений.	E3	Появляется процесс поддержания актуальности контекстного слоя (владелец, цикл обновления).
S2	Принципы поддержания актуальности общего контекстного слоя (owner, триггеры обновления).	Поддерживает актуальность контекстных документов команды на регулярной основе.	(якорь Сбера) Поддерживает актуальность документов, спецификаций, гайдлайнов, архитектурных решений команды/продукта.	E3-E4	Контекстный слой фактически повторно используется другими агентами/задачами/людьми.
S3	Принципы построения переиспользуемого контекстного слоя для множества задач/агентов.	Обеспечивает повторное использование контекстного слоя несколькими задачами/агентами без пересборки с нуля.	Контекстный слой используется минимум в 2 разных задачах/проектах без существенной переработки.	E4	— (высший уровень компетенции)

Закрашенные строки/ячейки — **якорные формулировки из исходной матрицы Сбера**; N0 — отсутствие покрытия.

2.4 Creating Tools

Разработка и интеграция tools в агентные пайплайны

Level	Знает	Умеет	Применяет	Evidence	Критерий перехода
N0	—	—	Программа не содержит работы с инструментами/коннекторами агента (MCP и аналоги).	—	Появление задания «подключи готовый коннектор/инструмент к агенту».
J1	Что такое инструмент/коннектор агента (MCP и аналоги) и зачем он нужен.	Подключает один готовый инструмент/коннектор к агенту по инструкции.	Выполняет задание с использованием подключённого готового инструмента.	E1	Появляется требование объяснить, какие данные/действия становятся доступны агенту через инструмент.
J2	Какие данные/действия открываются агенту через конкретный инструмент.	Объясняет назначение и зону действия подключённого инструмента.	(якорь Сбера, частично) Использует интеграцию готовых инструментов, данных и действий через MCP.	E1-E2	Появляется проверка, что инструмент не даёт доступа за пределы необходимого.
J3	Принцип минимально необходимых прав инструмента (связь с автономией агента).	Проверяет права/зону действия подключённого инструмента перед использованием в задаче.	(якорь Сбера) В результате агент может: получать дополнительный контекст, выполнять операции, обращаться к внешним системам — через осознанно подключённые интеграции.	E2	Появляется задача создания собственного инструмента под повторяющуюся задачу.
M1	Принципы создания простого инструмента (функция/скрипт/API-обёртка) для агента.	Разрабатывает простой инструмент под одну повторяющуюся задачу.	В проекте создаёт минимальный собственный инструмент, который агент вызывает как tool.	E2	Появляется документация инструмента, понятная агенту (описание, входы/выходы).
M2	(якорь Сбера) Принципы документирования инструмента для агента (описание назначения, входы/выходы).	Документирует созданный инструмент так, чтобы агент мог его корректно использовать.	(якорь Сбера) Пользователь умеет создавать новые инструменты, при этом агент успешно их использует.	E2-E3	Появляется тестирование инструмента на нескольких входных данных, включая граничные случаи.
M3	Принципы тестирования инструментов агента (happy path + граничные случаи).	Тестирует инструмент на нескольких входах, включая минимум один граничный случай.	Проводит >=3 тестовых вызова инструмента в >=2-3 проектах/итерациях.	E3	Появляется задача формирования реестра/библиотеки инструментов команды.
S1	Принципы построения реестра инструментов команды (название, назначение, владелец, статус).	Описывает созданные инструменты в едином реестре команды/программы.	Формирует структуру библиотеки инструментов для своей команды.	E3	Появляется процесс review перед публикацией инструмента в библиотеку.
S2	(якорь Сбера, усилено) Принципы review и публикации инструментов в командную библиотеку (аналог Pattern Library).	Проводит/проходит review инструмента перед добавлением в библиотеку.	(якорь Сбера) Создаёт командную библиотеку Tools с процессом приёма новых инструментов. ВАЖНО: «умеет сделать tool» недостаточно для S2 — нужно «создал reusable tooling, используемый повторно».	E3-E4	Минимум один инструмент из библиотеки используется другими членами команды без участия автора.
S3	Принципы поддержания переиспользуемости библиотеки инструментов (reusable tooling).	Обеспечивает использование своих инструментов другими без своего участия в подключении.	Минимум 1 инструмент библиотеки фактически переиспользован другим участником.	E4	— (высший уровень компетенции)

Закрашенные строки/ячейки — **якорные формулировки из исходной матрицы Сбера**; N0 — отсутствие покрытия.

2.5 Creating Skills

Адаптация агента и создание навыков

Level	Знает	Умеет	Применяет	Evidence	Критерий перехода
N0	—	—	Программа не содержит работы с навыками/skill-файлами агента.	—	Появление задания «используй существующий skill/шаблон в задаче».
J1	Что такое skill/навык агента (готовый шаблон/режим работы).	Использует существующий skill-файл/шаблон, выданный программой.	(якорь Сбера) Использует подсказки при написании кода; формулирует простые запросы с опорой на готовый skill.	E1	Появляется требование соблюдать ограничения, заданные skill/правилами, без отключения.
J2	(якорь Сбера) Принцип: ограничения работы агента (guardrails) не отключаются «для удобства».	Соблюдает заданные ограничения работы агента при использовании skill.	(якорь Сбера) Применяет правила и ограничения (rules) для работы агента в начале каждой сессии.	E1-E2	Появляется требование привести пример, где skill помог избежать ошибки (рефлексия).
J3	Типовые ошибки, которые предотвращает использование skill.	Приводит конкретный пример ситуации, где skill/подсказка предотвратила ошибку.	Очищает запросы от чувствительной информации; формулирует простые запросы с учётом skill.	E2	Появляется задача создания нового skill под класс задач.
M1	Принципы создания нового skill (специализированный режим работы под класс задач).	Создаёт новый skill/навык для агента под конкретный класс задач.	В проекте создаёт собственный skill и применяет его минимум один раз.	E2	Появляется сравнение результата «с навыком / без навыка».
M2	(якорь Сбера) Принципы проверки эффективности нового инструмента/навыка (сравнение результатов).	Сравнивает результат задачи с использованием нового skill и без него (хотя бы качественно).	(якорь Сбера) Пользователь умеет создавать новые инструменты, при этом агент успешно их использует — включая skills.	E2-E3	Появляется документация skill, достаточная для передачи другому человеку.
M3	Принципы документирования skill для передачи (назначение, условия применения, ограничения).	Документирует skill так, чтобы другой человек мог применить его без участия автора.	В >=2-3 проектах применяет документированные собственные skills.	E3	Появляется задача проверки качества skill перед публикацией (чек-лист/тестовые сценарии).
S1	Принципы минимальной проверки качества skill перед публикацией (аналог eval-проверки).	Формирует чек-лист/тестовые сценарии для проверки skill перед публикацией.	Проверяет собственный skill по чек-листу перед тем, как сделать его доступным другим.	E3	Появляется фактическое расширение возможностей агента подключением новых skills как стандартная практика.
S2	(якорь Сбера) Принципы расширения возможностей агента через подключение новых skills как системная практика.	(якорь Сбера) Умеет расширять возможности агента через подключение новых skills.	Систематически подключает/обновляет skills агента в проектах, ведёт версии.	E3-E4	Skill фактически переиспользуется другими людьми/командами без участия автора в адаптации.
S3	(якорь Сбера) Принципы поддержания reusable skills, востребованных другими (версионирование, эволюция).	(якорь Сбера) Созданные специалистом навыки регулярно переиспользуются (востребованы) другими.	Минимум 1 skill переиспользован >=2 другими пользователями; видна история версий с обоснованием изменений.	E4	— (высший уровень компетенции)

Закрашенные строки/ячейки — **якорные формулировки из исходной матрицы Сбера**; N0 — отсутствие покрытия.

2.6 Agent Teams

Управление командой агентов и субагентов

Level	Знает	Умеет	Применяет	Evidence	Критерий перехода
N0	—	—	Программа не содержит работы с субагентами/мультиагентными конфигурациями.	—	Появление задания «определи, какие агенты нужны для комплексной задачи».
J1	(якорь Сбера) Принцип декомпозиции комплексной задачи на подзадачи под разных агентов/субагентов.	Декомпозирует комплексную задачу на подзадачи.	(якорь Сбера) Определяет, какие агенты необходимы для решения комплексных задач.	E1	Появляется фактическое подключение хотя бы одного субагента.
J2	Как подключить субагента и передать ему подзадачу.	Подключает минимум одного субагента под выделенную подзадачу.	(якорь Сбера) Подключает субагентов для решения части комплексной задачи.	E1-E2	Появляется требование объяснить, почему именно эта подзадача делегирована отдельному агенту (изоляция контекста).
J3	Принцип изоляции контекста между основной задачей и субагентом.	Объясняет, почему подзадача делегирована отдельному агенту, а не выполнена в основной сессии.	Включает результат субагента в финальный результат основной задачи.	E2	Появляется запуск 2+ агентных сессий параллельно для разных подзадач одной задачи.
M1	Принципы параллельного запуска нескольких агентных сессий под разные подзадачи.	Запускает минимум 2 агентные сессии параллельно для разных подзадач одной задачи.	В проекте организует параллельные сессии под разные части задачи.	E2	Появляется способ свести результаты параллельных сессий в единый результат.
M2	Принципы сведения результатов параллельных сессий в единый результат без потери информации.	Сводит результаты параллельных сессий в единый результат (не «на глаз»).	Применяет единую схему сведения результатов в ≥ 1 проекте.	E2-E3	Появляется объяснение, как избегались конфликты/дублирование между параллельными сессиями, повторяется в нескольких проектах.
M3	(якорь Сбера) Принципы распределения задач между командами агентов в параллельных сессиях с учётом конфликтов/дублирования.	(якорь Сбера) Распределяет задачи между командами агентов в параллельных сессиях.	В ≥ 2 -3 проектах распределяет задачи между агентами параллельно, фиксируя механизм избежания конфликтов.	E3	Появляется конфигурация «оркестратор + рабочие агенты» для многоэтапной задачи.
S1	Принципы построения конфигурации «оркестратор + рабочие агенты» (orchestrator + workers) для многоэтапной задачи.	Выстраивает конфигурацию оркестратор + рабочие агенты для задачи с несколькими этапами.	Создаёт и использует такую конфигурацию в проекте.	E3	Появляются точки проверки результата перед передачей между агентами (мини human-in-the-loop Decision Map).
S2	Принципы установки точек проверки результата между агентами в оркестрации (мини Decision Map).	Устанавливает 1-2 ключевые точки проверки результата перед передачей между агентами.	(якорь Сбера, усилено) Управляет командой агентов и субагентов в агентном режиме, проверяет результат, корректирует решения от агента.	E3-E4	Итоговое решение о готовности результата принимается на основании собранных свидетельств (мини Evidence Bundle).
S3	Принципы формирования мини	Принимает итоговое решение	Демонстрирует схему оркестрации + набор	E4	— (высший уровень компетенции)

Level	Знает	Умеет	Применяет	Evidence	Критерий перехода
	Evidence Bundle для принятия итогового решения о результате команды агентов.	«принять/доработать» на основании собранного набора свидетельств (логи/результаты промежуточных шагов), а не «на глаз».	свидетельств + обоснованное итоговое решение.		

Закрашенные строки/ячейки — **якорные формулировки из исходной матрицы Сбера**; NO — отсутствие покрытия.

Часть 3. Competency Coverage Map (формат вывода сервиса)

Для каждой проанализированной программы сервис формирует таблицу:

Компетенция	Уровень	Комментарий
Prompting	M2	Сильная постановка задач и decomposition, но отсутствует reusable prompt system
Prompt Engineering	M1	Есть workflow prompting, но мало evidence reusable patterns
Context Management	S1	Сильный context package и спецификация
Creating Tools	J3	Используются интеграции, но нет собственного tooling
Creating Skills	N0	Практики reusable skills отсутствуют — нужна доработка для перехода к J1
Agent Teams	J1	Есть ознакомление с multi-agent logic

Правило определения уровня

Программе присваивается максимальный level, для которого выполнены критерии «Применяет» и достигнут целевой Evidence-уровень (на основе встречаемости в материалах программы: единичное упоминание ≈ E1–E2, повтор в нескольких проектах ≈ E3, явное требование переиспользования другими ≈ E4).

Если критерии level выполнены, но Evidence ниже целевого — присваивается level с пометкой «evidence не подтверждён» и рекомендацией, как его усилить (например, добавить требование повторного использования в чек-лист защиты).

N0 — обязательный комментарий

Для N0 обязателен комментарий по структуре методологии: что конкретно отсутствует и какое минимальное задание/требование закрывает переход к J1.

Расширенный комментарий к программе

Формируется как связный абзац (1–3 предложения) по образцу: профиль программы (например, «AI-native analyst / product profile»), сильные стороны (компетенции с высоким level и подтверждённым evidence), точки роста (компетенции N0/J1–J2 — что это ограничивает на выходе).

Пример: «Программа формирует strong AI-native analyst / product profile с сильным покрытием Context Management и Prompting. Недостаточно развиты Creating Skills и Agent Teams, поэтому программа пока не выводит обучающихся на уровень AI-native orchestration.»

Часть 4. Competency Profile человека (производный слой)

Та же сетка 6×10, но источник evidence — индивидуальные артефакты человека (логи, проекты, skills, инструменты), а не материалы программы. Используется, когда:

- сотрудник/студент проходит индивидуальную сертификацию;
- нужно сравнить «потенциал программы» (Coverage Map) с «фактическим результатом конкретного выпускника» (Profile) — например, программа даёт до M3 по Context Management, но конкретный выпускник предоставил evidence только на M1.

Статус: не реализуется в MVP — фиксируется как расширение после program-first MVP.

Часть 5. Market-facing statuses (план)

Внешние, понятные рынку формулировки уровня (для резюме/бейджей/сертификатов), производные от внутренних N0/J1–J3/M1–M3/S1–S3 — например, «AI-Augmented Practitioner», «Agentic Workflow Owner», «AI Orchestration Lead».

Статус: требует отдельной проработки нейминга и согласования со Сбером — не входит в MVP, фиксируется как backlog.

Часть 6. Internal Readiness Index (план)

Сводный индекс по программе/человеку, агрегирующий 6 компетенций в одну метрику готовности (например, взвешенная сумма достигнутых level с весами по компетенциям в зависимости от профиля программы).

Статус: требует решения о весах и формуле — backlog, не входит в MVP.

Ограничения v2 (требуют согласования с Сбером)

- 60 ячеек (6×10) — для MVP за 3 дня реалистично прогнать сервис по всем 60, но качество для «средних» подуровней (J2, M1, M2, S1, S2) при первом запуске следует трактовать как черновое — якорные формулировки Сбера покрывают только 6 из 60 ячеек напрямую, остальные 54 — авторская интерполяция.
- NO обязательный комментарий — реализуется как отдельное поле в выводе, не как «уровень с критериями».
- Evidence-модель (E1–E4) для программ оценивается приближённо: «повторяется в проектах» и «используют другие» для образовательной программы — это про требования программы к студенту (например, чек-лист защиты требует переиспользования другим студентом), а не про факт использования конкретным человеком. Это нормально для program-first логики, но стоит явно проговорить с методистами при первом тестовом прогоне.